

학급 관찰 일지 시스템 - AI 바ιβ 코딩 컨텍스트 문서

문서 목적: 학급 관찰 일지 시스템 개발을 위한 전체 요구사항 및 기술 사양 문서입니다.

작성자: 엄태주 (정보·컴퓨터 교사)

최종 수정: 2026-03-07

프로젝트 사이트: teaju.ai.kr

목차

- [1. 프로젝트 개요](#)
- [2. 핵심 사용 시나리오](#)
- [3. 기술 스택 및 제약사항](#)
- [4. 아키텍처 구조](#)
- [5. 데이터 모델 \(Firestore\)](#)
- [6. UI/UX 규칙](#)
- [7. 코딩 컨벤션](#)
- [8. 보안 및 성능 요구사항](#)
- [9. 개발 우선순위](#)
- [10. 에러 처리 및 예외 케이스](#)
- [11. 테스트 체크리스트](#)

프로젝트 개요

문제 정의

중학교 담임 교사는 학기 초 2주 내에 20~30명 학생의 성향을 파악해야 하지만, 수업 직후 관찰 내용을 기록할 시간이 부족하고, 수기/엑셀 기록은 검색 및 통계 분석이 어렵습니다.

솔루션

웹 기반 학급 관찰 일지 시스템으로, 교사가 수업 직후 1~2분 안에 학생별 태그와 메모를 기록하고, 학기 초 2주 내에 전체 학생 성향을 체크할 수 있습니다.

타겟 사용자

- 주 사용자: 중학교 담임 교사 (1인)
- 사용 환경: PC (Chrome/Edge), 스마트폰 (Android/iOS Safari)
- 디지털 리터러시: 중급 (구글 폼, 엑셀, 네이버 클라우드 정도는 익숙)

핵심 가치 제안

1. 빠른 기록: 수업 직후 1~2분 안에 관찰 내용 저장
2. 빠른 파악: 학기 초 2주 내 전체 학생 성향 체크 완료
3. 언제 어디서나: PC와 모바일에서 동일한 사용 경험
4. 데이터 기반: 태그 통계와 타임라인으로 학생 변화 추적

핵심 사용 시나리오

시나리오 A: 수업 직후 빠른 관찰 기록

상황: 3교시 수학 수업 직후, 복도에서 4교시 수업 장소로 이동 중 (1~2분 여유)

플로우:

1. 스마트폰으로 사이트 접속 (이미 로그인 상태)
2. 학생 카드 화면에서 "김철수" 카드의
태그 찍기
버튼 탭
3. "집중 좋음", "발표 적극" 태그 선택 →
저장
4. 같은 카드의
메모
버튼 탭
5. 교시: "3교시 수학", 내용: "어려운 문제도 끝까지 풀려고 시도함" 입력 →
저장
6. 완료 (총 소요 시간: 1분 30초)

기대 효과: 수업 중 관찰 내용을 잊어버리기 전에 즉시 기록

시나리오 B: 학기 초 2주 내 성향 체크

상황: 학기 시작 첫 주, 28명 학생 중 아직 5명의 성향을 체크하지 못함

플로우:

1. 점심시간 또는 방과 후, PC에서 접속
2. 우측 하단 플로팅 버튼

학기초 성향체크

클릭

3. 모달에서 미완료 학생 5명 중 "이영희" 선택
4. 체크리스트에서 "외향적", "친구 많음", "발표 선호", "과제 성실" 체크
5. 첫인상 메모: "밝고 활발함. 친구들과 잘 어울림" 입력
6. *저장 후 다음 학생*
클릭 → 자동으로 다음 미완료 학생으로 이동
7. 반복하여 5명 완료 (총 소요 시간: 10분)

기대 효과: 2주 내 전체 학생 성향 파악 완료, 상담 및 생활지도 시 참고

시나리오 C: 상담 전 학생 정보 확인

상황: 학부모 상담 전, 해당 학생의 최근 관찰 기록과 태그를 빠르게 훑어보기

플로우:

1. PC에서 접속, 상단 검색창에 "박민수" 입력
2. 박민수 학생 카드에서 현재 붙어있는 태그 확인: "산만함 (3회)", "과제 미제출 (2회)"
3. 카드 하단 최근 메모 미리보기 확인
4. (추후 구현) 카드 클릭 → 학생 상세 타임라인에서 날짜별 관찰 기록 전체 훑기

기대 효과: 상담 시 구체적인 사례 기반으로 대화 가능

기술 스택 및 제약사항

프론트엔드

- **HTML5**: 시맨틱 마크업 사용
- **Tailwind CSS (CDN)**: 버전 3.4.x 사용
 - CDN 링크: <https://cdn.tailwindcss.com>
 - 커스텀 색상은 `tailwind.config inline`으로 정의
- **Vanilla JavaScript (ES6+)**: 프레임워크 사용 금지 (React, Vue, Svelte 등 사용 안 함)
 - 이유: 바이브 코딩으로 한 파일에서 빠르게 프로토타입 제작 가능

백엔드

- **Firebase**:
 - **Authentication**: 구글 소셜 로그인만 지원
 - **Firestore**: NoSQL 데이터베이스
 - **Hosting**: 정적 호스팅
- **Firebase SDK**: CDN 버전 사용 (최신 버전 10.x 이상)
 - CDN 링크: Firebase 공식 CDN 사용
 - 모듈: `firebase-app`, `firebase-auth`, `firebase-firestore`

제약사항

1. **단일 HTML 파일 우선**: 초기 개발은 `index.html` 한 파일에 HTML + CSS + JS 모두 작성
 - 이유: 바이브 코딩 시 AI가 전체 맥락을 한 번에 이해 가능
 - 추후 필요시 파일 분리 가능 (`app.js`, `styles.css` 등)
 2. **외부 라이브러리 최소화**: jQuery, Lodash 등 사용 금지
 - 예외: Chart.js (통계 차트 구현 시 사용 가능)
 3. **이모지 사용 금지**: 모든 아이콘은 SVG 인라인 또는 Heroicons CDN 사용
 4. **브라우저 호환성**: Chrome/Edge 최신 버전, Safari 15+ (ES6+ 문법 사용 가능)
-

아키텍처 구조

파일 구조 (1차 개발 완료 시)

project-root/

- |—— index.html # 메인 HTML 파일 (CSS, JS 포함)
- |—— firebase-config.js # Firebase 설정 (gitignore)
- |—— .firebaserc # Firebase 프로젝트 설정
- |—— firebase.json # Hosting 설정
- |—— [README.md](#) # 프로젝트 설명

페이지/화면 구조

로그인 전 화면

- 중앙에 로고 + "학급 관찰 일지" 제목
- 구글 로그인

버튼 (Firebase Auth popup)

로그인 후 메인 화면 (학생 목록)

```
+-----+
| [로그] 학급 관찰 일지 [학년도] [학급] [+ 오늘 관찰] [사용자명 ▼] |
+-----+
| [사이드바] | [검색창: 학생 이름 검색] | | | | | | |
| - 학생 보기 | [필터] 전체(28) | 성향완료(23) | 주의필요(5) |
| - 통계/분석 | |
| - 설정 | [학생 카드 그리드] |
| | +-----+ +-----+ +-----+ |
| | | 김철수 | | 이영희 | | 박민수 | |
| | | 1번 | | 2번 | | 3번 | |
| | | [태그] | | [태그] | | [태그] | |
| | | [메모] | | [메모] | | [메모] | |
| | +-----+ +-----+ +-----+ |
| | |
| | [플로팅 버튼: 성향체크] |
+-----+
```

모바일 화면

- 상단 헤더 (sticky): 로고, 제목, 사용자명
- 메인 영역: 학생 카드 1열 세로 스크롤
- 하단 네비게이션 (fixed):

학생
통계
설정

- 우측 하단 플로팅 버튼:

학기 초 성향 체크

모달 종류

1. 태그 찍기 모달: 학생 선택 → 카테고리별 태그 버튼 → 저장
2. 메모 추가 모달: 학생 선택 → 교시/과목, 관찰 내용, 템플릿 선택 → 저장
3. 오늘 관찰 추가 모달: 학생 드롭다운 선택 → 교시/과목, 관찰 내용, 빠른 태그 → 저장
4. 학기 초 성향 체크 모달: 학생 그리드 선택 → 체크박스 리스트, 첫인상 메모 → 저장 후 다음 학생

데이터 모델 (Firestore)

컬렉션 구조 요약

컬렉션 이름	용도	주요 필드
users	교사 정보	email, name, school, createdAt
classes	학급 정보	teacherId, year, grade, classNumber, studentCount
students	학생 기본 정보	classId, name, number, gender, initialCheckCompleted
tags	학생별 태그 (누적)	studentId, category, label, count, isWarning
observations	관찰 메모	studentId, classId, date, time, subject, content
initialChecks	학기 초 성향 체크 결과	studentId, personality[], learning[], behavior[], firstImpression

상세 스키마

users 컬렉션

```
{
  userId: "auto-generated-id",
  email: "teacher@school.ac.kr",
  name: "염태주",
  school: "OO중학교",
  createdAt: Timestamp
}
```

classes 컬렉션

```
{
  classId: "auto-generated-id",
  teacherId: "userId",
  year: 2026,
  grade: 2,
  classNumber: 3,
  studentCount: 28,
}
```

```
createdAt: Timestamp
}
```

students 컬렉션

```
{
  studentId: "auto-generated-id",
  classId: "classId",
  name: "김철수",
  number: 1, // 출석번호
  gender: "남", // "남" 또는 "여"
  initialCheckCompleted: false, // 학기 초 성향 체크 완료 여부
  createdAt: Timestamp
}
```

정렬 규칙: number 필드 오름차순 (출석번호 순)

tags 컬렉션

```
{
  tagId: "auto-generated-id",
  studentId: "studentId",
  category: "personality", // "personality", "learning", "behavior"
  label: "외향적",
  count: 3, // 같은 태그를 찍은 횟수
  isWarning: false, // 주의(노란색) 또는 경고(빨간색) 태그 여부
  warningLevel: null, // null(일반), "caution"(주의), "alert"(경고)
  lastUpdated: Timestamp
}
```

태그 카테고리 및 기본 라벨:

- 성격/관계 (personality): 외향적, 내향적, 리더형, 따르는 편, 배려형, 친구 많음, 따로 노는 편, 조용함
- 학습 태도 (learning): 과제 성실, 개념 이해 빠름, 속도 느림, 실수 많음, 발표 선호, 발표 기피, 집중력 우수, 집중 어려움
- 생활/행동 (behavior):
 - 일반: 정리정돈 잘함, 복장 단정, 시간 잘 지킴
 - 주의 (warningLevel: "caution"): 산만함, 지각 잦음, 과제 미제출
 - 경고 (warningLevel: "alert"): 친구와 갈등

observations 컬렉션

```
{
  observationId: "auto-generated-id",
  studentId: "studentId",
  classId: "classId",
  date: "2026-03-07", // YYYY-MM-DD
  time: "14:30", // HH:mm
  subject: "3교시 수학", // 선택 항목
  content: "어려운 문제도 끝까지 풀려고 시도함",
  tags: [], // 관찰과 함께 찍은 태그 라벨 배열 (선택)
  createdAt: Timestamp
}
```

initialChecks 컬렉션

```
{
  checkId: "auto-generated-id",
  studentId: "studentId",
  classId: "classId",
  personality: ["외향적", "친구 많음"],
  learning: ["발표 선호", "과제 성실"],
  behavior: ["정리정돈 잘함"],
  firstImpression: "밝고 활발함. 친구들과 잘 어울림",
  createdAt: Timestamp
}
```

Firestore 쿼리 예시

학생 목록 불러오기 (출석번호 순)

```
const studentsRef = collection(db, 'students');
const q = query(
  studentsRef,
  where('classId', '==', currentClassId),
  orderBy('number', 'asc')
);
const querySnapshot = await getDocs(q);
```

특정 학생의 태그 불러오기

```
const tagsRef = collection(db, 'tags');
const q = query(
  tagsRef,
  where('studentId', '==', studentId)
);
const querySnapshot = await getDocs(q);
```

주의 필요 학생 필터링 (클라이언트 사이드)

```
// Firestore에서 모든 태그를 불러온 후, 클라이언트에서 필터링
const warningStudents = students.filter(student => {
  const studentTags = tags.filter(tag => tag.studentId ===
    student.studentId);
  const warningTags = studentTags.filter(tag => tag.warningLevel ===
    'caution' || tag.warningLevel === 'alert');
  const totalWarningCount = warningTags.reduce((sum, tag) => sum +
    tag.count, 0);
  return totalWarningCount >= 3;
});
```

UI/UX 규칙

색상 시스템

Primary 색상

- **메인:** #036242 (진한 녹색)
- **Hover:** #024e35 (더 진한 녹색)
- **Tailwind 클래스:** bg-[#036242], text-[#036242], hover:bg-[#024e35]

배경 및 기본 색상

- **배경:** #f8fafc (연한 회색) → bg-slate-50
- **카드:** #ffffff (흰색) → bg-white
- **텍스트 (검정):** #0f172a → text-slate-900
- **텍스트 (회색):** #64748b → text-slate-600

태그 색상 (Tailwind 클래스)

카테고리	배경 클래스	텍스트 클래스	용도
성격/관계	bg-sky-50	text-sky-700	외향적, 친구 많음 등
학습 태도	bg-indigo-50	text-indigo-700	과제 성실, 발표 선호 등
생활(일반)	bg-green-50	text-green-700	정리정돈 잘함 등
주의	bg-amber-50	text-amber-700	산만함, 지각 잦음 등
경고	bg-red-50	text-red-700	친구와 갈등

반응형 브레이크포인트

화면 크기	Tailwind 접두사	레이아웃 특징
모바일	~ md:	1열, 하단 네비게이션, 플로팅 버튼
태블릿	md: ~ lg:	2열 그리드
PC	lg:	3열 그리드, 좌측 사이드바

Tailwind 브레이크포인트:

- md: 768px 이상
- lg: 1024px 이상

타이포그래피

- 폰트: 시스템 기본 폰트 스택 사용

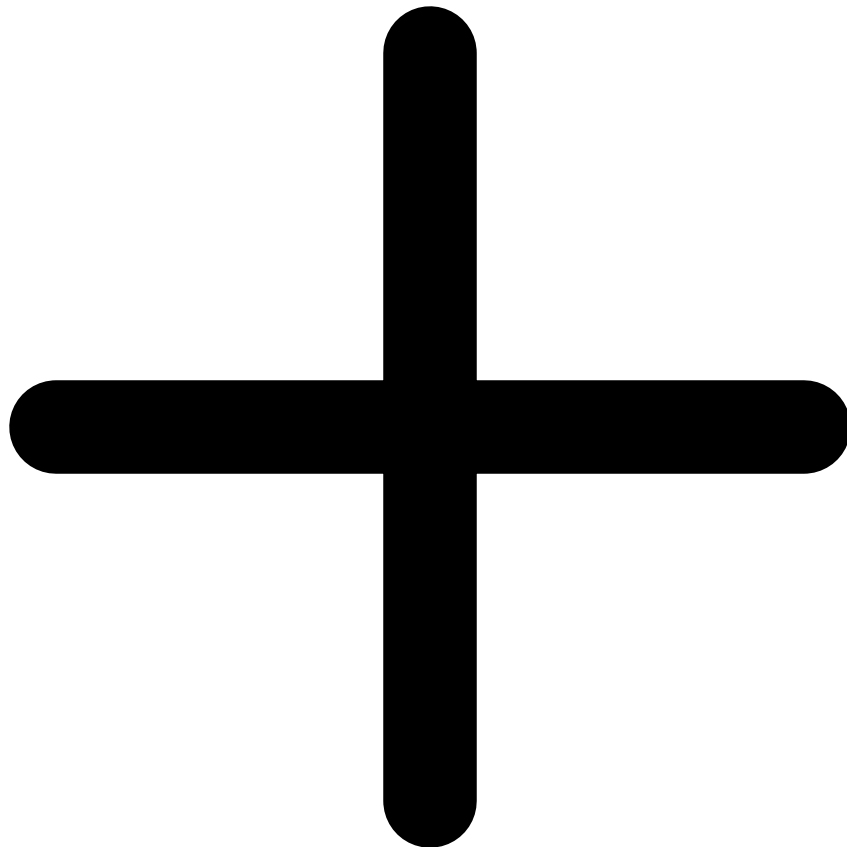
font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', 'Noto Sans KR', sans-serif;

- 크기:
 - 제목 (h1): text-2xl (24px) 또는 text-3xl (30px)
 - 부제 (h2): text-xl (20px)

- 본문: text-base (16px)
- 작은 텍스트: text-sm (14px)

아이콘 규칙

- **이미지 사용 금지:** 절대 ✕
- **영어 표기 금지:** 모든 UI 텍스트는 한글로만 작성 (버튼, 라벨, 메시지, 플레 이스홀더 등)
- **SVG 인라인:** 간단한 아이콘은 SVG 태그로 직접 작성



- **Heroicons:** 복잡한 아이콘은 Heroicons 사용

버튼 스타일

Primary 버튼

저장

Secondary 버튼 (테두리만)

취소

카드 스타일

모달 스타일

- **배경:** 반투명 검정 오버레이 (bg-black bg-opacity-50)
- **모달 박스:** 흰색 배경, 둥근 모서리, 그림자

모달 제목

취소 저장

애니메이션

- **카드 hover:** hover:shadow-md transition-shadow (살짝 올라가는 효과는 hover:-translate-y-1 transition-transform 추가)
- **모달 페이드 인:** CSS 또는 JS로 opacity 0 → 1
- **버튼 리플 효과:** 클릭 시 scale-95 → scale-100 트랜지션

접근성

- **키보드 접근:** 모든 버튼과 입력 필드는 Tab 키로 이동 가능
 - **ESC 키:** 모달 닫기
 - **포커스 스타일:** focus:ring-2 focus:ring-[#036242] focus:outline-none
 - **최소 터치 영역:** 모바일 버튼은 최소 44×44px
 - **한글 전용:** 모든 사용자 인터페이스 요소(버튼 텍스트, 라벨, 안내 메시지, 플레이스홀더, 에러 메시지 등)는 한글로만 작성하며, 영어 단어 사용 금지
-

코딩 컨벤션

JavaScript 스타일

- **ES6+ 문법 사용:** `const`, `let`, 화살표 함수, 템플릿 리터럴, 구조 분해 할당
- **함수명:** camelCase (`loadStudents`, `saveObservation`)
- **변수명:** camelCase, 의미 있는 이름 (`studentId`, `currentClassId`)
- **상수:** UPPER_SNAKE_CASE (`MAX_STUDENTS`, `DEFAULT_TAG_CATEGORIES`)

HTML 규칙

- **시맨틱 태그 사용:** `<header>`, `<main>`, `<section>`, `<article>`, `<footer>`
- **클래스명:** Tailwind 유틸리티 클래스 + 필요시 커스텀 클래스 (BEM 스타일)
 - 예: `student-card`, `tag-modal_header`

주석 규칙

- **함수 상단:** JSDoc 스타일

`/**`

- Firestore에서 학생 목록을 불러옵니다.
 - `@param {string} classId` - 학급 ID
 - `@returns {Promise<Array>}` 학생 객체 배열
- ```
*/
async function loadStudents(classId) {
 // ...
}
```

- **복잡한 로직:** 한 줄 주석으로 설명

```
// 주의/경고 태그 카운트 합산 후 3회 이상이면 필터링
const warningCount = tags.filter(t => t.warningLevel).reduce((sum, t)
=> sum + t.count, 0);
```

## 파일 구조 (추후 분리 시)

```
project-root/
├── index.html
├── css/
│ └── styles.css
├── js/
│ ├── app.js # 메인 로직
│ ├── firebase.js # Firebase 초기화 및 인증
│ ├── students.js # 학생 CRUD
│ ├── tags.js # 태그 CRUD
│ └── observations.js # 관찰 메모 CRUD
├── assets/
└── logo.svg
```

---

## 보안 및 성능 요구사항

### Firebase Security Rules

```
rules_version = '2';
service cloud.firestore {
 match /databases/{database}/documents {
```

```
// 인증된 사용자만 접근 가능
match /{document=**} {
 allow read, write: if request.auth != null;
}

// users 컬렉션: 본인 문서만 읽기/쓰기
match /users/{userId} {
 allow read, write: if request.auth.uid == userId;
}

// classes 컬렉션: 담임 교사만 접근
match /classes/{classId} {
 allow read, write: if request.auth.uid == resource.data.teacherId;
}
```

```

// students 컬렉션: 해당 학급의 담임만 접근
match /students/{studentId} {
 allow read, write: if request.auth.uid == get(/databases/$(database)/document/{documentId}).auth.uid;
}

// tags, observations, initialChecks: 동일한 로직
match /tags/{tagId} {
 allow read, write: if request.auth.uid == get(/databases/$(database)/document/{documentId}).auth.uid;
}

match /observations/{observationId} {
 allow read, write: if request.auth.uid == get(/databases/$(database)/document/{documentId}).auth.uid;
}

match /initialChecks/{checkId} {
 allow read, write: if request.auth.uid == get(/databases/$(database)/document/{documentId}).auth.uid;
}
}
}

```

**주의:** Security Rules는 Firebase Console에서 직접 설정하거나, `firestore.rules` 파일로 배포

## 개인정보 보호

- **HTTPS 필수:** Firebase Hosting은 기본 HTTPS 제공
- **학생 개인정보:** 이름, 성별, 출석번호, 관찰 내용은 민감 정보
  - 외부 유출 방지: Security Rules로 접근 제한
  - 백업 권장: Firebase Console에서 정기 백업 설정
- **데이터 이관:** 교사 전근 시 다른 교사에게 학급 데이터 이관 방법 필요 (추후 구현)



## 성능 최적화

### 1. Firestore 쿼리 최적화:

- 필요한 필드만 가져오기 (불가능한 경우 클라이언트 필터링)
- 페이지네이션: 학생 수가 50명 이상이면 `limit(30)` + "더 보기" 버튼

### 2. 실시간 리스너 최소화:

- 학생 목록 화면에만 실시간 리스너 사용
- 모달에서는 일회성 쿼리 (`getDocs`) 사용

### 3. 이미지 최적화:

- SVG 아이콘 사용 (용량 작음)
- 추후 학생 사진 추가 시 WebP 포맷, 200×200px 이하로 리사이징

### 4. 로딩 상태 표시:

- Firestore 쿼리 중 스피너 표시
- 저장 성공/실패 시 토스트 알림 (3초 자동 사라짐)

---

## 개발 우선순위

### Phase 1: 기본 구조 및 Firebase 연동 [P1]

**목표:** 로그인 후 학생 목록 표시까지 구현

1. Firebase 프로젝트 생성 및 설정
2. HTML 기본 레이아웃 (헤더, 사이드바, 메인 영역)
3. Tailwind CSS 적용 (CDN, 색상 커스터마이징)
4. Firebase 초기화 (Authentication, Firestore)
5. 구글 로그인 기능 (`signInWithPopup`)
6. 로그인 상태 확인 (`onAuthStateChanged`)
7. 로그인 전/후 화면 분기
8. 사용자 정보 Firestore에 저장 (`users` 컬렉션)

**검증:** 구글 로그인 성공 → 메인 화면 진입 → Firebase Console에서 `users` 문서 확인

## Phase 2: 학생 관리 [P1]

**목표:** 학생 목록 표시 및 카드 렌더링

1. `students` 컬렉션에서 데이터 불러오기 (Firestore 쿼리)
2. 학생 카드 HTML 동적 생성 (이름, 출석번호, 성별, 아바타)
3. 그리드 레이아웃 적용 (PC: 3열, 태블릿: 2열, 모바일: 1열)
4. 카드 `hover` 애니메이션
5. 검색 기능 (상단 검색창, 실시간 필터링)
6. 필터 버튼 (전체, 성향 체크 완료, 주의 필요)

**검증:** Firebase Console에서 테스트 학생 데이터 추가 → 화면에 카드 표시 → 검색/필터 동작 확인

## Phase 3: 태그 시스템 [P1]

**목표:** 태그 찍기 및 표시

1. "태그 찍기" 버튼 클릭 시 모달 열기
2. 모달 UI (카테고리별 태그 버튼, 커스텀 태그 입력)
3. 태그 선택/해제 토글 (배경색 변경)
4. 저장 시 `Firestore tags` 컬렉션에 저장 (신규 추가 또는 `count` 증가)
5. 학생 카드에 태그 로드 및 표시 (색상, `count`)
6. 태그 기반 필터링 (주의 필요 학생)

**검증:** 학생 카드에서 태그 찍기 → Firebase Console 확인 → 카드에 태그 표시 → 필터링 동작

## Phase 4: 관찰 메모 [P1]

**목표:** 메모 작성 및 미리보기

1. "메모" 버튼 클릭 시 모달 열기
2. 입력 항목: 교시/과목, 관찰 내용, 템플릿 선택
3. 템플릿 드롭다운 → `textarea` 자동 입력
4. 저장 시 `Firestore observations` 컬렉션에 저장
5. 학생 카드에 최근 메모 미리보기 표시 (최대 2줄)

**검증:** 메모 작성 → Firebase Console 확인 → 카드에 미리보기 표시

## Phase 5: 학기 초 성향 체크 [P2]

**목표:** 성향 체크 모달 및 진행 상황 추적

1. 플로팅 버튼 (모바일 우측 하단, PC는 사이드바) 클릭 시 모달 열기
2. 학생 선택 그리드 (3~4열, 선택 시 테두리 강조)
3. 진행 상황 표시 ("체크 완료: X명 / 미완료: Y명")
4. 체크리스트 (카테고리별 체크박스)
5. 첫인상 메모 textarea
6. 저장 시:
  - Firestore initialChecks 컬렉션에 저장
  - 체크한 항목들을 tags 컬렉션에도 저장
  - students.initialCheckCompleted → true
7. "저장 후 다음 학생" 버튼 → 다음 미완료 학생으로 자동 이동

**검증:** 성향 체크 → Firebase Console 확인 → 진행률 업데이트 → 필터 "성향 체크 완료" 동작

## Phase 6: 통계 및 대시보드 [P3 - 추후 구현]

**목표:** 데이터 시각화 및 분석

1. 대시보드 화면 추가 (사이드바 "통계/분석" 메뉴)
2. 가장 많이 붙은 태그 TOP 5 (막대 그래프)
3. 주의 필요 학생 리스트
4. 성향 체크 진행률 (진행 바)
5. 최근 7일 관찰 기록 수 (선 그래프)
6. (선택) Chart.js 라이브러리 사용

**검증:** 대시보드 진입 → 차트 표시 → 데이터 정확성 확인

## Phase 7: 고급 기능 [P4 - 미래 고려사항]

- 학생별 타임라인 (날짜순 관찰 기록)
  - PDF/엑셀 내보내기
  - 학부모 연락 기록
  - 자리 배치도 뷰
  - 음성 메모 기능
-

# 에러 처리 및 예외 케이스

## Firestore 연동 에러

로그인 실패

케이스: 구글 로그인 팝업 취소 또는 실패

처리:

```
try {
 const result = await signInWithPopup(auth, provider);
} catch (error) {
 if (error.code === 'auth/popup-closed-by-user') {
 // 사용자가 팝업을 닫음 → 아무 동작 안 함
 } else {
 // 토스트 알림: "로그인에 실패했습니다. 다시 시도해주세요."
 showToast('로그인에 실패했습니다. 다시 시도해주세요.', 'error');
 }
}
```

## Firestore 읽기/쓰기 실패

케이스: 네트워크 오류, Security Rules 거부

처리:

```
try {
 const querySnapshot = await getDocs(q);
} catch (error) {
 console.error('Firestore 읽기 실패:', error);
 showToast('데이터를 불러오는데 실패했습니다. 새로고침 후 다시 시도해주세요.', 'error');
}
```

## 사용자 입력 검증

필수 입력 항목 누락

케이스: 관찰 메모에서 "관찰 내용" 없이 저장 시도

처리:

```
if (!content.trim()) {
 showToast('관찰 내용을 입력해주세요.', 'warning');
 return;
}
```

중복 태그 방지

케이스: 같은 태그를 동시에 여러 번 찍으려고 할 때

처리: Firestore 트랜잭션 사용 또는 클라이언트 사이드 중복 체크

```
const existingTag = tags.find(tag => tag.studentId === studentId &&
tag.label === label);
if (existingTag) {
 // count 증가
 await updateDoc(doc(db, 'tags', existingTag.tagId), {
 count: increment(1),
 lastUpdated: serverTimestamp()
 });
} else {
 // 신규 추가
 await addDoc(collection(db, 'tags'), { ... });
}
```

네트워크 오류

케이스: 오프라인 상태에서 접속 시도

처리:

- Firestore 오프라인 지속성 활성화 (선택)
- 네트워크 오류 감지 시 화면 상단에 배너 표시: "인터넷 연결을 확인해주세요."

```
window.addEventListener('offline', () => {
 showBanner('인터넷 연결이 끊어졌습니다. 일부 기능이 제한될 수 있습니다.');
```

```
'warning');
});

window.addEventListener('online', () => {
 hideBanner();
});
```

## UI/UX 예외 처리

### 빈 데이터 상태

**케이스:** 학생이 한 명도 없을 때

**처리:** 빈 상태 메시지 표시

등록된 학생이 없습니다.

관리자가 학생 정보를 추가할 때까지 기다려주세요.

### 검색 결과 없음

**케이스:** 검색어에 해당하는 학생이 없을 때

**처리:** "검색 결과가 없습니다." 메시지 표시

---

## 테스트 체크리스트

### 기능 테스트

#### Phase 1: 인증

- [ ] 구글 로그인 버튼 클릭 시 팝업 열림
- [ ] 로그인 성공 시 메인 화면으로 이동
- [ ] 로그인 실패 시 에러 토스트 표시
- [ ] 로그아웃 버튼 클릭 시 로그인 화면으로 돌아감
- [ ] 새로고침 시 로그인 상태 유지 (세션 유지)

## Phase 2: 학생 관리

- ☐ 학생 목록이 출석번호 순으로 정렬되어 표시됨
- ☐ 학생 카드에 이름, 출석번호, 성별이 표시됨
- ☐ PC에서 3열, 태블릿에서 2열, 모바일에서 1열 그리드 표시
- ☐ 검색창에 학생 이름 입력 시 실시간 필터링 동작
- ☐ 필터 버튼 (전체, 성향 완료, 주의 필요) 클릭 시 필터링 동작

## Phase 3: 태그 시스템

- ☐ "태그 찍기" 버튼 클릭 시 모달 열림
- ☐ 태그 버튼 클릭 시 선택/해제 토글 (배경색 변경)
- ☐ 커스텀 태그 추가 기능 동작
- ☐ 저장 버튼 클릭 시 Firestore에 저장되고 모달 닫힘
- ☐ 학생 카드에 태그가 표시되고 count가 정확함
- ☐ 주의/경고 태그는 노란색/빨간색으로 표시됨

## Phase 4: 관찰 메모

- ☐ "메모" 버튼 클릭 시 모달 열림
- ☐ 템플릿 선택 시 textarea에 자동 입력됨
- ☐ 필수 입력(관찰 내용) 없이 저장 시도 시 경고 토스트 표시
- ☐ 저장 버튼 클릭 시 Firestore에 저장되고 모달 닫힘
- ☐ 학생 카드에 최근 메모 미리보기가 표시됨 (최대 2줄)

## Phase 5: 학기 초 성향 체크

- ☐ 플로팅 버튼 클릭 시 모달 열림
- ☐ 학생 선택 시 테두리 강조
- ☐ 진행 상황 표시가 정확함 (완료/미완료 학생 수)
- ☐ 체크박스 선택 후 저장 시 Firestore에 저장됨
- ☐ "저장 후 다음 학생" 클릭 시 다음 미완료 학생으로 이동
- ☐ 성향 체크 완료한 학생은 `initialCheckCompleted: true`로 업데이트됨

## 반응형 테스트

- ☐ PC (1920×1080): 3열 그리드, 좌측 사이드바 표시
- ☐ 태블릿 (768×1024): 2열 그리드, 하단 네비게이션 표시
- ☐ 모바일 (375×667): 1열 리스트, 하단 네비게이션, 플로팅 버튼 표시
- ☐ 가로/세로 모드 전환 시 레이아웃 깨지지 않음

## 브라우저 호환성

- ☐ Chrome 최신 버전
- ☐ Edge 최신 버전
- ☐ Safari 15+ (macOS, iOS)

## 성능 테스트

- ☐ 학생 30명 기준 초기 로딩 시간 3초 이내
- ☐ 태그 찍기, 메모 저장 시 1초 이내 완료
- ☐ 검색 입력 시 실시간 필터링 지연 없음

## 보안 테스트

- ☐ 로그인하지 않은 상태에서 Firestore 접근 시도 → 거부됨
- ☐ 다른 교사의 학급 데이터 접근 시도 → 거부됨 (Security Rules 테스트)

---

## 용어 사전 (Glossary)

| 한글 용어    | 영문 용어         | 설명                      |
|----------|---------------|-------------------------|
| 학급       | Class         | 특정 학년도의 특정 학년-반         |
| 학생       | Student       | 학급에 속한 개별 학생            |
| 태그       | Tag           | 학생 성향/행동을 나타내는 라벨       |
| 관찰 메모    | Observation   | 특정 날짜/시간에 기록한 학생 관찰 내용  |
| 성향 체크    | Initial Check | 학기 초 학생 성향을 빠르게 파악하는 기능 |
| 출석번호     | Number        | 학생의 고유 번호 (1~30번 등)     |
| 주의/경고 태그 | Warning Tag   | 생활지도가 필요한 행동을 나타내는 태그   |

---



# 변경 이력 (Changelog)

## v1.0 (2026-03-07)

- 초기 컨텍스트 문서 작성
- 프로젝트 개요, 기술 스택, 데이터 모델, UI/UX 규칙 정의
- Phase 1~6 개발 우선순위 명시
- 에러 처리 및 테스트 체크리스트 추가

## v1.1 (2026-03-07)

- 바이브 코딩 관련 내용 제거 (AI 사용법 가이드 삭제)
- 영어 표기 금지 규칙 추가 (모든 UI는 한글 전용)

---

## 문의 및 지원

- 개발자: 엄태주 (정보·컴퓨터 교사)
- 프로젝트 사이트: [teaju.ai.kr](http://teaju.ai.kr)
- 개발 방식: 바이브 코딩 (AI 보조 개발)

---

이 문서는 학급 관찰 일지 시스템의 전체 요구사항과 기술 사양을 정의합니다. 개발 시 이 문서의 규칙과 제약사항을 준수하여 일관성 있는 시스템을 구축하시기 바랍니다.